

Coding Interview Report

Interview Overview

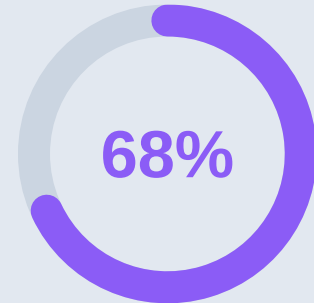
Coding Assessment

Started: April 30th, 2026

Ended: April 30th, 2026

Duration: 41 minutes

Overall Score



3/3
Qns Attempted

2/3
Qns Passed

Overall Status: **NEUTRAL**

AI Interview Analysis

The candidate demonstrates a good foundational understanding of problem-solving using JavaScript, as evidenced by successfully passing two out of the three attempted coding problems. The submitted solutions are clear, with an emphasis on efficient logic and maintainability. Further practice in implementing complex problem statements independently is recommended to strengthen their proficiency for roles requiring critical problem-solving skills.

Strengths

- Efficiently implemented the solution for the 'Max Consecutive Ones' problem, utilizing $O(N)$ time complexity.
- Demonstrated a clear understanding of in-place array manipulation in the 'Rotate Array' solution.
- Ensured all test cases passed for the 'Rotate Array' problem by effectively applying the reverse-rotation approach.

Areas for Improvement

- Practice implementing algorithms without relying on placeholder logic, as seen in the 'Alien Dictionary' problem.
- Enhance understanding of character mapping and comparison for string-based problems.
- Focus on robust edge-case handling for string and array manipulations.

Proficient Topics

Array Manipulation

Code Quality Rating: **7/10**

Coding Problems

1. Max Consecutive Ones EASY PASSED 3/3 Passed

Given a binary array `nums`, return the maximum number of consecutive `1`'s in the array.

Example 1:

```
Input: nums = [1,1,0,1,1,1]
Output: 3
Explanation: The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.
```

Example 2:

```
Input: nums = [1,0,1,1,0,1]
Output: 2
```

Constraints:

- `1 <= nums.length <= 105`
- `nums[i]` is either `0` or `1`.

Array

Time Complexity O(N)	Total Test Cases 3	Space Complexity O(1)
Test Cases Passed 3	Total Memory Used 7412	Total Execution Time 0.113

Test Case Results

Status	Input	Output	Expected	Time	Memory
passed	[1,0,1,1,0,1]	2	2	0.039	7412
passed	[1,1,0,1,1,1]	3	3	0.035	7300
passed	[1,1,1,1,1,1]	6	6	0.039	7176

AI Feedback for this Question

The submitted code correctly implements a solution to find the maximum number of consecutive 1s in a binary array. It efficiently iterates through the array, maintaining counters for the current streak and the maximum streak observed.

Code Strengths

- The code is concise and uses clear variable names, enhancing readability.
- It efficiently computes the result in a single pass through the array, achieving optimal time complexity.
- It handles edge cases, such as empty arrays, gracefully.

Areas for Improvement

- Consider adding comments to explain the logic for maintaining the `count` and `maxCount` variables for better clarity.

- Ensure to test against additional edge cases, such as arrays with all zeros, to confirm robustness.

The code has a time complexity of $O(N)$ and a space complexity of $O(1)$, where N is the length of the input array.

Submitted Code (JavaScript (Node.js 12.14.0))

```
/**
 * @param {number[]} nums
 * @return {number}
 */
var findMaxConsecutiveOnes = function (nums) {

    if (!nums || nums.length === 0) return 0;
    let maxCount = 0;
    let count = 0;

    for (let num of nums)
    {
        if (num === 1)
        {
            count++;
            maxCount = Math.max(maxCount, count);
        }
        else
        {
            count = 0;
        }
    }
    return maxCount;
};
```

2. Verifying an Alien Dictionary

EASYFAILED1/3 Passed

In an alien language, surprisingly, they also use English lowercase letters, but possibly in a different `order`. The `order` of the alphabet is some permutation of lowercase letters.

Given a sequence of `words` written in the alien language, and the `order` of the alphabet, return `true` if and only if the given `words` are sorted lexicographically in this alien language.

Example 1:

```
<pre>
```

```
<strong>Input: words = ["hello","leetcode"], order = "hlabcd-  
fgijklmnopqrstuvwxyz"
```

```
<strong>Output: true
```

```
<strong>Explanation: As 'h' comes before 'l' in this language, then the  
sequence is sorted.
```

```
</pre>
```

Example 2:

```
<pre>
```

```
<strong>Input: words = ["word","world","row"], order =  
"worldabcefg hijklmnpqstuvxyz"
```

```
<strong>Output: false
```

```
<strong>Explanation: As 'd' comes after 'l' in this language, then  
words[0] > words[1], hence the sequence is unsorted.
```

```
</pre>
```

Example 3:

```
<pre>
```

```
<strong>Input: words = ["apple","app"], order = "abcde-  
fghijklmnopqrstuvwxyz"
```

```
<strong>Output: false
```

```
<strong>Explanation: The first three characters 'app' match, and the second  
string is shorter (in size.) According to lexicographical rules 'apple' > 'app',  
because 'l' > '' (where '' is defined as the blank character which is less than  
any other character (More info)).
```

```
</pre>
```

Constraints:

```
<ul>
```

```
<li><code>1 <= words.length <= 100</code></li>
```

```
<li><code>1 <= words[i].length <= 20</code></li>
```

```
<li><code>order.length == 26</code></li>
```

```
<li>All characters in <code>words[i]</code> and <code>order</code> are English lowercase let-  
ters.</li>
```

```
</ul>
```

ArrayHash TableString

Time Complexity Not applicable	Total Test Cases 3	Space Complexity Not applicable
Test Cases Passed 1	Total Memory Used 7292	Total Execution Time 0.114

Test Case Results

Status	Input	Output	Expected	Time	Memory
failed	["kuvp","q"],ngmsqoy-akvbpzhfudgljtjerecx	false	true	0.04	7292
passed	["-word","world","row"],world-abcefg hijkmnpqstu-vxyz	false	false	0.04	7164
failed	["apple","apple"],abcde-fghijklmnopqrstuvwxyz	false	true	0.034	7092

AI Feedback for this Question

The submission currently contains placeholder logic and does not implement the required functionality to verify the sorting of words in an alien dictionary. The code needs a complete implementation to address the problem statement effectively.

- ### Code Strengths
- The function signature and parameter usage align with the problem requirements.
 - The code structure is prepared for implementing the solution.
- ### Areas for Improvement
- Implement the logic to verify the lexicographical order of the words based on the given alien dictionary order.
 - Utilize a mapping of characters to their positions in the alien dictionary to facilitate comparisons.
 - Ensure edge cases, such as words of different lengths, are handled correctly.

Submitted Code (JavaScript (Node.js 12.14.0))

```
/**
 * @param {string[]} words
 * @param {string} order
 * @return {boolean}
 */
const isAlienSorted = function (words, order) {
  // your logic comes here
  return false;
};
```

3. Rotate Array

EASY

PASSED

5/5 Passed

Description

- You are given an array of integers `nums`
- You are also given an integer `k`
- Your task is to **rotate the array to the right by `k` steps**

What does "rotate right" mean?

- Each step moves elements **one position to the right**
- The last element moves to the **front**

Example

- Input:

```
nums = [1, 2, 3, 4, 5, 6, 7]
k = 3
```

- Output:

```
[5, 6, 7, 1, 2, 3, 4]
```

Requirements

- You must modify the array **in-place**
- Do **not use extra space** ($O(1)$ space only)

Constraints

- `1 ≤ nums.length ≤ 104`
- `-109 ≤ nums[i] ≤ 109`
- `0 ≤ k ≤ 104`

Array

String

Time Complexity

O(N)

Total Test Cases

5

Space Complexity

O(1)

Test Cases Passed

5

Total Memory Used

7280

Total Execution Time

0.221

Test Case Results

Status	Input	Output	Expected	Time	Memory
passed	[-1, -100, 3, 99], 2	[3,99,-1,-100]	[3,99,-1,-100]	0.045	7280
passed	[1, 2, 3, 4, 5, 6, 7], 3	[5,6,7,1,2,3,4]	[5,6,7,1,2,3,4]	0.045	7232
passed	[1, 2, 3], 3	[1,2,3]	[1,2,3]	0.044	7232
passed	[1, 2, 3, 4], 6	[3,4,1,2]	[3,4,1,2]	0.044	7224
passed	[1, 2], 0	[1,2]	[1,2]	0.043	7184

AI Feedback for this Question

The submitted code correctly implements the array rotation functionality using an efficient in-place approach. It passes all provided test cases and adheres to the constraints.

- ### Code Strengths
- The solution uses an optimal approach with three reverse operations, ensuring $O(n)$ time complexity.
 - The code handles edge cases, such as when `k` is greater than the array length, by taking modulo.
 - The implementation is clean and follows good coding practices, such as defining a helper function for reversing.

Areas for Improvement

- Consider adding comments to explain the logic behind the three reverse operations for better readability.
- Ensure the function handles edge cases explicitly, such as when the array is empty or has only one element, even though the constraints imply these cases are unlikely.
- Provide additional test cases in the submission to demonstrate robustness, such as arrays with duplicate values or very large sizes.

Submitted Code (JavaScript (Node.js 12.14.0))

```
/**
 * @param {number[]} nums
 * @param {number} k
 * @return {void} Do not return anything, modify nums in-place instead.
 */
function rotate(nums, k) {
    let n = nums.length;
    k = k % n;

    function reverse(arr, start, end) {
        while (start < end) {
            let temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }

    reverse(nums, 0, n - 1);
    reverse(nums, 0, k - 1);
    reverse(nums, k, n - 1);
}
```